# 🔍 SawaFlix Sprint 1 Backend Code Review

**Date:** February 21, 2026
**Review Period:** Sprint 1 - Creator Verification System
**Presentation Date:** Tuesday, February 24, 2026 @ 10:00 AM
**Reviewers:** Backend Team Lead

---

## 📋 Executive Summary

This comprehensive code review covers **two critical Sprint 1 backend tasks**:

1. **[Wohking's Task]** - Build Verification API & Supabase Storage
2. **[Ngam's Task]** - Extend Auth Roles & Implement OTP Security

### ✅ Overall Status: PARTIAL - NEEDS CRITICAL FIXES

- **Code Quality:** 65/100
- **Security:** 55/100
- **Production Readiness:** 45/100
- **Deliverability:** NOT READY for 10 AM Tuesday unless critical issues are fixed

---

## 🎯 TASK 1: Build Verification API & Supabase Storage (Wohking)

### Current Implementation

- ✅ `POST /api/verification/submit` - created
- ✅ `PUT /api/verification/draft` - created
- ✅ `POST /api/upload` - file upload with type detection
- ✅ Zod validation for categories
- ✅ Bearer token authentication support

## 🔴 CRITICAL ISSUES

### Issue #1: Missing File Size Validation

**Severity:** HIGH
**Location:** `app/api/upload/route.ts`

**Current Code:**

```
const file = formData.get("file") as File | null;
const validation = UploadSchema.safeParse({ category: categoryInput });
if (!file || !validation.success) {
  return NextResponse.json(
    { error: "Invalid file or category" },
    { status: 400 },
  );
}
```

**Problem:** No file size check before processing. A malicious user could upload 1GB+ files, causing:

- Storage quota overflow
- Memory exhaustion
- Denial of Service (DoS)

**Fix Required:**

```
// Add these constants at the top
const MAX_FILE_SIZE = 50 * 1024 * 1024; // 50MB per specification
const ALLOWED_MIME_TYPES = [
  'image/jpeg', 'image/png', 'application/pdf',
  'image/webp', 'video/mp4', 'video/webm'
];

// Add size validation after file retrieval
if (file.size > MAX_FILE_SIZE) {
  return NextResponse.json(
    { error: `File size exceeds ${MAX_FILE_SIZE / 1024 / 1024}MB limit` },
    { status: 413 }
  );
}

// Add MIME type validation
if (!ALLOWED_MIME_TYPES.includes(type.mime)) {
  return NextResponse.json(
    { error: `File type ${type.mime} not allowed` },
    { status: 400 }
  );
}
```

---

## Issue #2: Missing Rate Limiting on Upload Endpoint

**Severity:** HIGH
**Location:** `app/api/upload/route.ts`

**Problem:** No rate limiting means rapid-fire uploads could:

- Exhaust storage quota
- Trigger billing spikes
- Enable abuse attacks

**Solution:**

```
import { redis, rateLimit } from "@/lib/redis";

export async function POST(req: Request) {
  // ... existing code ...

  if (isServiceRole) {
    creator_id = "admin-tester";
  } else {
    // Add rate limiting
    const { success } = await rateLimit.limit(`upload_limit:${creator_id}`);
    if (!success) {
      return NextResponse.json(
        { error: "Upload limit exceeded. Max 10 uploads per hour." },
        { status: 429 }
      );
    }
  }
  // ... rest of code ...
}
```

**Note:** Update rate limit in `lib/redis.ts`:

```
export const rateLimit = new Ratelimit({
  redis: redis,
  limiter: Ratelimit.slidingWindow(10, "1 h"), // 10 uploads per hour
});
```

---

## Issue #3: No Storage RLS (Row-Level Security) Policy Defined

**Severity:** CRITICAL
**Location:** Database Configuration (Missing)

**Problem:**

- No RLS policies configured for `verification-docs` bucket
- Any authenticated user could read/delete other users' documents
- Admin access not properly scoped

**Required Database Migration:**

```
-- Create bucket with proper policies
INSERT INTO storage.buckets (id, name, public)
VALUES ('verification-docs', 'verification-docs', false)
ON CONFLICT (id) DO NOTHING;

-- Policy 1: Users can upload only to their own folder
CREATE POLICY "Users can upload to own folder"
ON storage.objects FOR INSERT
WITH CHECK (
```

```
  auth.uid()::text = (storage.foldername(name))[1]
  AND bucket_id = 'verification-docs'
);

-- Policy 2: Users can read own files
CREATE POLICY "Users can read own files"
ON storage.objects FOR SELECT
USING (
  auth.uid()::text = (storage.foldername(name))[1]
  AND bucket_id = 'verification-docs'
);

-- Policy 3: Admins can read all verification documents
CREATE POLICY "Admins can read all verification docs"
ON storage.objects FOR SELECT
USING (
  bucket_id = 'verification-docs'
  AND (
    SELECT has_admin_role(auth.uid()) -- function must exist
  )
);

-- Policy 4: No one can delete unless admin
CREATE POLICY "Admins can delete verification docs"
ON storage.objects FOR DELETE
USING (
  bucket_id = 'verification-docs'
  AND (SELECT has_admin_role(auth.uid()))
);
```

---

## Issue #4: Missing Database Table Definition

**Severity:** CRITICAL
**Location:** Database Schema (Not Found)

**Problem:** No clear schema definition for `verification_submissions` table. The code assumes it exists but:

- Column names might be inconsistent
- No constraints defined
- Missing indexes for performance

**Required Migration:**

```
CREATE TABLE public.verification_submissions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  creator_id UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
  category VARCHAR(50) NOT NULL CHECK (
    category IN ('traditional_stories', 'music', 'food', 'other')
  ),
  form_data JSONB NOT NULL DEFAULT '{}',
  status VARCHAR(20) NOT NULL DEFAULT 'unverified' CHECK (
    status IN ('unverified', 'pending', 'approved', 'rejected', 'changes_requested')
```

```
  ),
  admin_notes TEXT,
  reviewed_at TIMESTAMP WITH TIME ZONE,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),

  -- Constraints
  UNIQUE(creator_id) -- One active submission per creator
);

-- Indexes for performance
CREATE INDEX idx_verification_status ON verification_submissions(status);
CREATE INDEX idx_verification_creator ON verification_submissions(creator_id);
CREATE INDEX idx_verification_created_at ON verification_submissions(created_at DESC);

-- Enable RLS
ALTER TABLE verification_submissions ENABLE ROW LEVEL SECURITY;

-- RLS Policies
CREATE POLICY "Creators can read own submissions"
ON verification_submissions FOR SELECT
USING (creator_id = auth.uid());

CREATE POLICY "Admins can read all submissions"
ON verification_submissions FOR SELECT
USING (
  (SELECT role FROM public.profiles WHERE id = auth.uid()) = 'admin'
);

CREATE POLICY "Creators can insert own submissions"
ON verification_submissions FOR INSERT
WITH CHECK (creator_id = auth.uid());

CREATE POLICY "Admins can update submissions"
ON verification_submissions FOR UPDATE
USING (
  (SELECT role FROM public.profiles WHERE id = auth.uid()) = 'admin'
);
```

---

## Issue #5: Service Role Key Misuse in Upload Logic

**Severity:** MEDIUM
**Location:** `app/api/upload/route.ts`, lines 27-34

**Problem:**

```
const expectedHeader = `Bearer ${serviceKey}`;
const isServiceRole = authHeader === expectedHeader;
```

This is **dangerous**:

1. Service role key is sensitive - shouldn't be used in client headers

2. The check uses simple string comparison (brittle)
3. No JWT validation or expiration check

**Better Approach:**

```
export async function POST(req: Request) {
  const cookieStore = await cookies();
  const supabase = createRouteHandlerClient({ cookies: () => cookieStore });

  // Get authenticated user from session (not Bearer tokens)
  const { data: { user }, error: authError } = await supabase.auth.getUser();

  if (authError || !user) {
    return NextResponse.json(
      { error: "Unauthorized" },
      { status: 401 }
    );
  }

  const creator_id = user.id;

  // ... proceed with upload using authenticated user
}
```

---

## Issue #6: No Virus/Malware Scanning

**Severity:** HIGH
**Location:** `app/api/upload/route.ts`

**Problem:** Files are uploaded directly without scanning for:

- Malware
- Suspicious code
- Exploits

**Solution (ClamAV Integration):**

```
import NodeClam from 'clamscan';

const NodeClamOptions = {
  clamdscan: {
    host: process.env.CLAMAV_HOST || 'localhost',
    port: parseInt(process.env.CLAMAV_PORT || '3310'),
  },
};

const clamscan = await new NodeClam().init(NodeClamOptions);

// After file retrieval, before upload:
const { isInfected, viruses } = await clamscan.scanBuffer(buffer);
```

```
if (isInfected) {
  console.error('🚨 MALWARE DETECTED:', viruses);
  return NextResponse.json(
    { error: "File failed security scan" },
    { status: 400 }
  );
}
```

**Note:** ClamAV requires Docker deployment or cloud service (VirusTotal API as alternative).

---

## Issue #7: Missing File Integrity Checks (Checksums)

**Severity:** MEDIUM
**Location:** `app/api/upload/route.ts`

**Problem:** No way to verify file wasn't corrupted during upload

**Solution:**

```
import crypto from 'crypto';

const buffer = Buffer.from(await file.arrayBuffer());
const fileHash = crypto.createHash('sha256').update(buffer).digest('hex');

// Store hash in verification_submissions.form_data
const { data, error: uploadError } = await adminClient.storage
  .from("verification-docs")
  .upload(filePath, buffer, {
    contentType: type.mime,
    metadata: { hash: fileHash } // Store hash
  });

// Later, retrieve and verify:
const storedFile = await adminClient.storage
  .from("verification-docs")
  .download(filePath);

const retrievedHash = crypto
  .createHash('sha256')
  .update(Buffer.from(await storedFile.data.arrayBuffer()))
  .digest('hex');

if (fileHash !== retrievedHash) {
  throw new Error("File integrity check failed");
}
```

---

### Issue #8: Incomplete Error Handling

**Severity:** MEDIUM
**Location:** `app/api/upload/route.ts`, line 80

**Current:**

```
} catch (err: any) {
  console.error("Critical Error:", err.message);
  return NextResponse.json(
    { error: "Internal Server Error" },
    { status: 500 },
  );
}
```

**Problem:** Generic error message doesn't help debugging. Also exposes stack traces.

**Better:**

```
} catch (err: any) {
  console.error('Upload error details:', {
    message: err.message,
    code: err.code,
    timestamp: new Date().toISOString(),
  });

  // Return safe error message
  const isValidationError = err.message?.includes('validation');
  return NextResponse.json(
    {
      error: isValidationError ? err.message : "Upload failed. Please try again.",
      code: isValidationError ? 'VALIDATION_ERROR' : 'UPLOAD_ERROR'
    },
    { status: isValidationError ? 400 : 500 }
  );
}
```

---

## 🟡 MEDIUM PRIORITY ISSUES

### Issue #9: Missing Logging for Audit Trail

**Severity:** MEDIUM
**Location:** `app/api/upload/route.ts`

**Problem:** No audit trail for file uploads makes it impossible to:

- Track who uploaded what
- Debug issues
- Detect suspicious patterns

**Solution:**

```
// Add function at top of file
async function logUploadEvent(
  creator_id: string,
  category: string,
  fileName: string,
  fileSize: number,
  status: 'success' | 'failed',
  error?: string
) {
  try {
    const adminClient = createClient(
      process.env.NEXT_PUBLIC_SUPABASE_URL!,
      process.env.SUPABASE_SERVICE_ROLE_KEY!
    );

    await adminClient
      .from('upload_logs')
      .insert({
        creator_id,
        category,
        file_name: fileName,
        file_size: fileSize,
        status,
        error_message: error,
        ip_address: req.headers.get('x-forwarded-for') || 'unknown',
        created_at: new Date().toISOString(),
      });
  } catch (logErr) {
    console.error('Failed to log upload event:', logErr);
  }
}

// Call in POST handler:
await logUploadEvent(creator_id, validation.data.category, fileName, file.size, 'success');
```

---

## Issue #10: Missing Concurrent Upload Limit

**Severity:** MEDIUM
**Location:** `app/api/upload/route.ts`

**Problem:** Multiple concurrent uploads could cause race conditions or exceed system resources

**Solution:**

```
import pLimit from 'p-limit';

// At module level
const uploadLimit = pLimit(3); // Max 3 concurrent uploads per user

export async function POST(req: Request) {
  // ... auth code ...
```

```
  const uploadPromise = uploadLimit(async () => {
    // Existing upload logic
    const { data, error: uploadError } = await adminClient.storage
      .from("verification-docs")
      .upload(filePath, buffer, { contentType: type.mime });
  });

  const { data, error: uploadError } = await uploadPromise;
  // ... rest of code ...
}
```

---

## ✅ WHAT'S WORKING WELL

1. **Zod Validation** - Good schema validation for category
2. **File Type Detection** - Using `fileTypeFromBuffer` is secure
3. **Bearer Token Support** - Good for testing, but needs refinement
4. **Clear Code Structure** - Well-organized try-catch blocks

---

# 🎯 TASK 2: Extend Auth Roles & Implement OTP Security (Ngam)

## Current Implementation

- ✅ OTP generation (6-digit code)
- ✅ Redis storage with 5-min expiration
- ✅ OTP verification logic
- ✅ Rate limiting on OTP send
- ❌ Missing database schema (`role`, `verification_status`)
- ❌ Middleware checks incomplete

## 🔴 CRITICAL ISSUES

### Issue #11: Database Schema NOT Updated

**Severity:** CRITICAL
**Location:** Database Schema (Missing)

**Problem:** Sprint requirements state:

> "Add role and verification_status to the Supabase profiles table"

**Current Status:** NOT DONE

- Current code uses `is_verified` (boolean) from `users` table
- Missing `role` column (enum: 'viewer', 'creator', 'admin')
- Missing `verification_status` column (enum: 'unverified', 'pending', 'approved', 'rejected')

**Required Migration:**

```
-- 1. Add columns to profiles table
ALTER TABLE public.profiles
ADD COLUMN IF NOT EXISTS role VARCHAR(20) DEFAULT 'viewer' CHECK (
  role IN ('viewer', 'creator', 'admin')
),
ADD COLUMN IF NOT EXISTS verification_status VARCHAR(20) DEFAULT 'unverified' CHECK (
  verification_status IN ('unverified', 'pending', 'approved', 'rejected',
'changes_requested')
),
ADD COLUMN IF NOT EXISTS is_verified BOOLEAN DEFAULT FALSE; -- Keep for backward compat

-- 2. Create a state machine function to enforce valid transitions
CREATE OR REPLACE FUNCTION verify_status_transition(
  old_status VARCHAR,
  new_status VARCHAR,
  user_role VARCHAR
) RETURNS BOOLEAN AS $$
BEGIN
  -- Only admins and the system can change status
  -- Valid transitions:
  -- unverified -> pending (user submits form)
  -- pending -> approved/rejected (admin reviews)
  -- approved/rejected -> pending (user resubmits)

  IF old_status IS NULL THEN
    RETURN new_status IN ('unverified', 'pending');
  END IF;

  CASE
    WHEN old_status = 'unverified' AND new_status = 'pending' THEN RETURN TRUE;
    WHEN old_status = 'pending' AND new_status IN ('approved', 'rejected',
'changes_requested') THEN RETURN TRUE;
    WHEN old_status IN ('approved', 'rejected') AND new_status = 'pending' THEN RETURN TRUE;
    ELSE RETURN FALSE;
  END CASE;
END;
$$ LANGUAGE plpgsql;

-- 3. Add trigger to prevent invalid transitions
CREATE OR REPLACE FUNCTION check_status_transition()
RETURNS TRIGGER AS $$
BEGIN
  IF NOT verify_status_transition(OLD.verification_status, NEW.verification_status,
NEW.role) THEN
    RAISE EXCEPTION 'Invalid verification status transition: % -> %',
      OLD.verification_status, NEW.verification_status;
  END IF;

  -- Sync is_verified with status for backward compatibility
  NEW.is_verified := (NEW.verification_status = 'approved');
```

```
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trigger_verify_status_transition
BEFORE UPDATE ON public.profiles
FOR EACH ROW
EXECUTE FUNCTION check_status_transition();

-- 4. Update RLS policies
ALTER TABLE public.profiles ENABLE ROW LEVEL SECURITY;

DROP POLICY IF EXISTS "Users can view own profile" ON public.profiles;
DROP POLICY IF EXISTS "Users can update own profile" ON public.profiles;

CREATE POLICY "Users can view own profile"
ON public.profiles FOR SELECT
USING (auth.uid() = id);

CREATE POLICY "Users can update own profile"
ON public.profiles FOR UPDATE
USING (auth.uid() = id)
WITH CHECK (
  -- Users can only update their own data, not sensitive fields
  -- Admins are handled separately
  (auth.uid() = id AND role = 'viewer') OR
  (SELECT role FROM public.profiles WHERE id = auth.uid()) = 'admin'
);

CREATE POLICY "Admins can view and update all profiles"
ON public.profiles FOR ALL
USING ((SELECT role FROM public.profiles WHERE id = auth.uid()) = 'admin');
```

---

### Issue #12: OTP Rate Limiting Too Restrictive

**Severity:** MEDIUM
**Location:** `app/api/otp/send/route.ts` & `lib/redis.ts`

**Current:**

```
export const rateLimit = new Ratelimit({
  redis: redis,
  limiter: Ratelimit.slidingWindow(3, "1 h"), // 3 requests per HOUR
})
```

**Problem:**

- User can only request 3 OTPs per hour
- If they mistype email or hit button accidentally, they're locked out for 57 minutes
- Too harsh for legitimate users

**Recommended Fix:**

```
// In lib/redis.ts - Create separate rate limiters
export const otpSendRateLimit = new Ratelimit({
  redis: redis,
  limiter: Ratelimit.slidingWindow(5, "15 m"), // 5 attempts per 15 minutes
});

export const otpVerifyRateLimit = new Ratelimit({
  redis: redis,
  limiter: Ratelimit.slidingWindow(10, "1 h"), // 10 verification attempts per hour
});

// In app/api/otp/send/route.ts
const { success } = await otpSendRateLimit.limit(`otp_send:${email}`);
if (!success) {
  return NextResponse.json(
    { error: "Too many OTP requests. Try again in 15 minutes." },
    { status: 429 }
  );
}

// In app/api/otp/verify/route.ts
const { success } = await otpVerifyRateLimit.limit(`otp_verify:${email}`);
if (!success) {
  return NextResponse.json(
    { error: "Too many verification attempts. Try again later." },
    { status: 429 }
  );
}
```

---

## Issue #13: Missing OTP Transport (Email/SMS Not Actually Sent)

**Severity:** CRITICAL
**Location:** `app/api/otp/send/route.ts`

**Current Code:**

```
export async function POST(req: Request) {
  // ... validation code ...
  const otp = Math.floor(100000 + Math.random() * 900000).toString();
  await redis.set(`otp:${email}`, otp, { ex: 300})
  console.log(`sending otp to ${email} with code ${otp}`) // ❌ JUST LOGS IT!
  return NextResponse.json({ message: "OTP sent successfully" })
}
```

**Problem:**

- OTP is **logged to console**, not actually sent
- Frontend shows "sending" but nothing reaches user
- **Users can't actually verify**

**Solution - Using SendGrid Email:**

```
import sgMail from '@sendgrid/mail';
import crypto from 'crypto';

sgMail.setApiKey(process.env.SENDGRID_API_KEY!);

export async function POST(req: Request) {
    try {
        const { email } = await req.json();

        // Validation
        if (!email) {
            return NextResponse.json(
                { error: "Email is required" },
                { status: 400 }
            );
        }

        // Rate limiting
        const { success } = await otpSendRateLimit.limit(`otp_send:${email}`);
        if (!success) {
            return NextResponse.json(
                { error: "Too many OTP requests. Try again in 15 minutes." },
                { status: 429 }
            );
        }

        // Generate OTP
        const otp = Math.floor(100000 + Math.random() * 900000).toString();

        // Store in Redis with 5-minute expiration
        await redis.set(`otp:${email}`, otp, { ex: 300 });

        // Send via SendGrid
        await sgMail.send({
            to: email,
            from: process.env.SENDGRID_FROM_EMAIL!,
            subject: '🔐 Your SawaFlix Verification Code',
            html: `
                <div style="font-family: Arial, sans-serif; max-width: 600px; margin: 0
auto;">
                    <h2>Welcome to SawaFlix</h2>
                    <p>Your verification code is:</p>
                    <h1 style="color: #FF6B35; font-size: 48px; letter-spacing: 5px; margin:
20px 0;">
                        ${otp}
                    </h1>
                    <p style="color: #666; font-size: 14px;">
                        This code expires in 5 minutes.
                    </p>
                    <p style="color: #999; font-size: 12px;">
                        Don't share this code with anyone.
```

```
                    </p>
                </div>
            `,
        });

        // Log for audit
        console.log(`[OTP Sent] Email: ${email}, Timestamp: ${new Date().toISOString()}`);

        return NextResponse.json({
            message: "OTP sent to your email successfully",
            // Don't expose OTP in response in production
            debug: process.env.NODE_ENV === 'development' ? otp : undefined
        });

    } catch (error) {
        console.error('Error sending OTP:', error);

        // Don't leak internal error details
        if (error instanceof Error && error.message.includes('Invalid email')) {
            return NextResponse.json(
                { error: "Invalid email address" },
                { status: 400 }
            );
        }

        return NextResponse.json(
            { error: 'Failed to send OTP. Please try again.' },
            { status: 500 }
        );
    }
}
```

**Environment Variables Needed:**

```
SENDGRID_API_KEY=sg_xxxxx
SENDGRID_FROM_EMAIL=verify@sawaflix.com
```

---

## Issue #14: OTP Verification Missing Rate Limiting

**Severity:** HIGH
**Location:** `app/api/otp/verify/route.ts`

**Current Code:**

```
export async function POST(req: Request) {
    try {
        const { email, code } = await req.json();
        const storedOtp = await redis.get(`otp:${email}`)

        if (!storedOtp) {
            return NextResponse.json({error: `OTP exprired or invalid` }, { status: 400 })
```

```
        }
        if (String(storedOtp).trim() !== String(code).trim()) {
            return NextResponse.json({ error: `Invalid OTP` }, { status: 400 }) // ❌ No
rate limit
        }
        // ... rest of code
    }
}
```

**Problem:** No rate limiting = brute force attacks possible (attacker could try 1,000,000 codes)

**Fix:**

```
import { otpVerifyRateLimit } from '@/lib/redis';

export async function POST(req: Request) {
    try {
        const { email, code } = await req.json();

        if (!email || !code) {
            return NextResponse.json(
                { error: "Email and code are required" },
                { status: 400 }
            );
        }

        // Rate limit verification attempts
        const { success } = await otpVerifyRateLimit.limit(`otp_verify:${email}`);
        if (!success) {
            return NextResponse.json(
                { error: "Too many verification attempts. Try again in 1 hour." },
                { status: 429 }
            );
        }

        const storedOtp = await redis.get(`otp:${email}`);

        if (!storedOtp) {
            return NextResponse.json(
                { error: "OTP expired or not found. Request a new code." },
                { status: 400 }
            );
        }

        if (String(storedOtp).trim() !== String(code).trim()) {
            // Don't delete OTP yet - let it expire naturally
            return NextResponse.json(
                { error: "Invalid OTP" },
                { status: 400 }
            );
        }

        // Successful verification - delete OTP
```

```
        await redis.del(`otp:${email}`);

        // Update user verification status
        const { error } = await supabaseAdmin
            .from('profiles')
            .update({
                is_verified: true,
                verification_status: 'approved'
            })
            .eq('email', email);

        if (error) {
            console.error('Error updating user verification status:', error);
            return NextResponse.json(
                { error: 'Verification failed. Please try again.' },
                { status: 500 }
            );
        }

        return NextResponse.json({
            message: 'User verified successfully',
            verified: true
        });

    } catch (error) {
        console.error('Error verifying OTP:', error);
        return NextResponse.json(
            { error: 'Server error' },
            { status: 500 }
        );
    }
}
```

---

## Issue #15: Middleware Not Checking User Role

**Severity:** HIGH
**Location:** `middleware.js`

**Current Logic:**

```
const { data: profile } = await supabase
  .from('users')
  .select('is_verified')
  .eq('id', user.id)
  .single()

if (profile && profile.is_verified) {
  isVerified = true
}

if (!isVerified) {
  if (isProtectedRoute) {
```

```
    return NextResponse.redirect(new URL('/verify-otp', request.url))
  }
}
```

**Problems:**

1. Checks `users` table but requirements specify `profiles` table
2. Only checks `is_verified`, doesn't check role
3. Creator route `/creator/verify` should only be accessible to creators with role='creator'
4. Doesn't enforce verification_status state machine

**Improved Middleware:**

```
import { NextResponse } from 'next/server'
import { createClient } from './utils/supabase/middleware'

const PROTECTED_ROUTES = {
  creator: ['/creator', '/dashboard'],
  admin: ['/admin'],
  verified: ['/dashboard'] // requires is_verified=true
}

export async function middleware(request) {
  const { supabase, response } = createClient(request)

  try {
    await supabase.auth.getSession()
    const { data: { user } } = await supabase.auth.getUser()
    const { pathname } = request.nextUrl

    // No user - redirect to login
    if (!user) {
      const publicRoutes = ['/', '/login', '/sign-up', '/sign-in']
      if (!publicRoutes.includes(pathname)) {
        const redirectUrl = new URL('/login', request.url)
        redirectUrl.searchParams.set('redirectedFrom', pathname)
        return NextResponse.redirect(redirectUrl)
      }
      return response
    }

    // User logged in - fetch their profile
    const { data: profile, error: profileError } = await supabase
      .from('profiles')
      .select('
        id,
        role,
        verification_status,
        is_verified,
        created_at
      ')
      .eq('id', user.id)
```

```
      .single()

    if (profileError || !profile) {
      console.error('Profile fetch error:', profileError)
      return NextResponse.redirect(new URL('/login', request.url))
    }

    // Check if user is trying to access creator routes
    if (pathname.startsWith('/creator')) {
      if (profile.role !== 'creator') {
        return NextResponse.redirect(new URL('/', request.url))
      }
      // Creator can access verification portal at any status
      if (pathname === '/creator/verify') {
        return response
      }
      // But full creator dashboard requires approval
      if (profile.verification_status !== 'approved') {
        return NextResponse.redirect(new URL('/creator/verify', request.url))
      }
    }

    // Check admin routes
    if (pathname.startsWith('/admin')) {
      if (profile.role !== 'admin') {
        return NextResponse.redirect(new URL('/', request.url))
      }
      return response
    }

    // Check general dashboard access
    if (pathname.startsWith('/dashboard')) {
      if (!profile.is_verified || profile.verification_status !== 'approved') {
        return NextResponse.redirect(new URL('/verify-otp', request.url))
      }
    }

    // Redirect verified users away from auth pages
    if (['/login', '/sign-up', '/sign-in'].includes(pathname)) {
      if (profile.role === 'admin') {
        return NextResponse.redirect(new URL('/admin', request.url))
      }
      if (profile.role === 'creator' && profile.verification_status === 'approved') {
        return NextResponse.redirect(new URL('/creator/dashboard', request.url))
      }
      if (profile.is_verified) {
        return NextResponse.redirect(new URL('/dashboard', request.url))
      }
      if (profile.verification_status === 'pending') {
        return NextResponse.redirect(new URL('/creator/verify', request.url))
      }
    }
```

```
    return response

  } catch (error) {
    console.error('Middleware error:', error)
    return response
  }
}

export const config = {
  matcher: [
    '/((?!_next/static|_next/image|favicon.ico|.*\\.(?:svg|png|jpg|jpeg|gif|webp)$).*)',
  ],
}
```

---

### Issue #16: Missing Admin Role Assignment Logic

**Severity:** MEDIUM
**Location:** Database & Auth Actions (Not Found)

**Problem:**

- No process to assign `admin` role
- Requirements say admins must exist but how do they get created?
- Current signup only creates 'viewer' or 'creator' roles

**Solution:**

```
-- Create stored procedure for admin assignment (protected with RLS)
CREATE OR REPLACE FUNCTION assign_admin_role(target_user_id UUID)
RETURNS TABLE(success BOOLEAN, message TEXT) AS $$
BEGIN
  -- Only existing admins can assign admin roles
  IF (SELECT role FROM profiles WHERE id = auth.uid()) != 'admin' THEN
    RETURN QUERY SELECT FALSE, 'Only admins can assign admin role'::TEXT;
    RETURN;
  END IF;

  UPDATE profiles
  SET role = 'admin'
  WHERE id = target_user_id;

  RETURN QUERY SELECT TRUE, 'Admin role assigned successfully'::TEXT;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;
```

**First Admin Setup (One-time, manual):**

```
-- After initial admin user signup, run this once:
UPDATE public.profiles
SET role = 'admin'
WHERE email = 'admin@sawaflix.com';
```

**Documentation Needed:** Add note in backend guide explaining admin onboarding process.

---

## Issue #17: Verification Status Transitions Not Enforced

**Severity:** HIGH
**Location:** Database (No state machine)

**Problem:**

- Code assumes valid status transitions but doesn't enforce them
- A bug could allow invalid state: rejected → approved without pending
- No audit trail of who changed status and when

**Partially Fixed By:** Issue #11's migration with trigger and state machine function

---

## Issue #18: No Session Invalidation After Status Changes

**Severity:** MEDIUM
**Location:** Admin approval endpoints

**Current** `app/api/admin/verifications/approve/route.ts`**:**

```
// 2. Flip the verification switch in Profiles
await supabase.from("profiles").update({ is_verified: true }).eq("id", target_creator_id);
```

**Problem:**

- User still logged in with old token showing unverified status
- Requires full page refresh to see updated status
- Poor UX

**Solution:**

```
// After approval, create session refresh token
async function invalidateUserSession(userId: string) {
  // Send event through Supabase Realtime to notify client
  const supabase = createClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.SUPABASE_SERVICE_ROLE_KEY!
  );

  // Broadcast to user's active sessions
  await supabase
    .channel(`user:${userId}`)
    .send({
      type: 'broadcast',
      event: 'profile_updated',
      payload: { action: 'refresh_required' }
    });
}
```

```
// In approve endpoint:
await supabase.from("profiles").update({
  is_verified: true,
  verification_status: 'approved'
}).eq("id", target_creator_id);

// Notify user to refresh
await invalidateUserSession(target_creator_id);
```

**Frontend listener (in layout.tsx):**

```
useEffect(() => {
  const channel = supabase
    .channel(`user:${user?.id}`)
    .on('broadcast', { event: 'profile_updated' }, (payload) => {
      if (payload.payload.action === 'refresh_required') {
        // Trigger profile refresh and UI update
        window.location.href = '/dashboard'; // Simple refresh
      }
    })
    .subscribe();

  return () => channel.unsubscribe();
}, [user?.id]);
```

---

## 🟡 MEDIUM PRIORITY ISSUES

### Issue #19: No OTP Expiration Warning

**Severity:** LOW
**Location:** `app/api/otp/send/route.ts`

**Problem:** Users don't know when OTP expires. No countdown.

**Solution:** Include expiration time in response:

```
return NextResponse.json({
  message: "OTP sent successfully",
  expiresIn: 300, // seconds
  expiresAt: new Date(Date.now() + 300000).toISOString()
});
```

---

### Issue #20: Console Logs in Production

**Severity:** MEDIUM
**Location:** Multiple files

**Current:**

```
console.log(`Verifying OTP for ${email} with code ${code}, storeddd OTP is ${storedOtp}`)
console.log(`sending otp to ${email} with code ${otp}`)
```

**Problem:**

- Logs OTP codes which could be visible in log aggregation services
- Performance overhead
- Security risk

**Solution:**

```
// Create logging utility
export const safeLog = (level: 'info' | 'warn' | 'error', message: string, data?: any) => {
  if (process.env.NODE_ENV === 'production') {
    // Only log errors and warnings in production, no data
    if (level !== 'info') {
      console.error(`[${level.toUpperCase()}] ${message}`);
    }
  } else {
    // Detailed logging in development
    console[level](`[${level.toUpperCase()}] ${message}`, data);
  }
};

// Usage:
safeLog('info', 'OTP verification requested', { email: maskedEmail(email) });
```

---

## ✅ WHAT'S WORKING WELL (Ngam's Task)

1. **Redis Integration** - Good use of Upstash
2. **OTP Generation** - Secure 6-digit generation
3. **5-Minute Expiration** - Correct security timing
4. **Try-Catch Blocks** - Good error handling structure

---

# 📊 Cross-Task Issues

## Issue #21: No Integration Tests Between API Endpoints

**Severity:** MEDIUM
**Location:** Test Suite (Missing)

**Problem:**

- File upload works in isolation
- Verification API works in isolation
- But does the full flow work? (signup → generate OTP → upload files → submit → admin approve)

**Solution - Create Integration Test Suite:**

```
// __tests__/verification-flow.integration.test.ts
import { describe, it, expect, beforeAll, afterAll } from '@jest/globals';
import { createClient } from '@supabase/supabase-js';

describe('Creator Verification Flow - End to End', () => {
  let testUserId: string;
  let testEmail: string = `test_${Date.now()}@sawaflix.com`;
  let otp: string;

  it('Step 1: User signs up as creator', async () => {
    const response = await fetch('/api/auth/signup', {
      method: 'POST',
      body: JSON.stringify({
        email: testEmail,
        password: 'Test@1234',
        category: 'creator',
        phone: '+237670000000'
      })
    });
    expect(response.status).toBe(200);
    const data = await response.json();
    testUserId = data.user.id;
  });

  it('Step 2: OTP is sent', async () => {
    const response = await fetch('/api/otp/send', {
      method: 'POST',
      body: JSON.stringify({ email: testEmail })
    });
    expect(response.status).toBe(200);
  });

  it('Step 3: OTP is verified', async () => {
    // In test, retrieve OTP from Redis directly
    const response = await fetch('/api/otp/verify', {
      method: 'POST',
      body: JSON.stringify({
        email: testEmail,
        code: otp // Retrieved from test Redis
      })
    });
    expect(response.status).toBe(200);
  });

  it('Step 4: File is uploaded', async () => {
    const file = new File(['test'], 'test.pdf', { type: 'application/pdf' });
    const formData = new FormData();
    formData.append('file', file);
    formData.append('category', 'national_id');

    const response = await fetch('/api/upload', {
      method: 'POST',
      body: formData,
```

```
      headers: { 'Authorization': `Bearer ${testToken}` }
    });
    expect(response.status).toBe(200);
  });

  it('Step 5: Verification is submitted', async () => {
    const response = await fetch('/api/verification/submit', {
      method: 'POST',
      body: JSON.stringify({
        category: 'music',
        form_data: { /* wizard data */ }
      }),
      headers: { 'Authorization': `Bearer ${testToken}` }
    });
    expect(response.status).toBe(200);
    const data = await response.json();
    expect(data.status).toBe('pending');
  });

  it('Step 6: Admin approves creator', async () => {
    const response = await fetch('/api/admin/verifications/approve', {
      method: 'POST',
      body: JSON.stringify({
        target_creator_id: testUserId,
        notes: 'Approved in test'
      }),
      headers: { 'Authorization': `Bearer ${adminToken}` }
    });
    expect(response.status).toBe(200);
  });
});
```

---

## Issue #22: No Error Recovery Documentation

**Severity:** LOW
**Location:** Documentation (Missing)

**Problem:**

- What if upload fails halfway?
- Can user resume?
- What about draft autosave?

**Solution:** Add recovery guide to docs/backend_guide.md

---

# 🔐 Security Recommendations (Both Tasks)

## 1. HTTPS Enforcement

```
// Add to next.config.ts
module.exports = {
  headers: async () => [
    {
      source: '/:path*',
      headers: [
        { key: 'Strict-Transport-Security', value: 'max-age=31536000; includeSubDomains' },
        { key: 'X-Content-Type-Options', value: 'nosniff' },
        { key: 'X-Frame-Options', value: 'DENY' },
        { key: 'X-XSS-Protection', value: '1; mode=block' },
        { key: 'Referrer-Policy', value: 'strict-origin-when-cross-origin' },
      ],
    },
  ],
};
```

## 2. CORS Configuration

```
// Create api/middleware.ts
import { NextRequest, NextResponse } from 'next/server';

export function corsMiddleware(req: NextRequest) {
  const origin = req.headers.get('origin');
  const allowedOrigins = [
    'https://sawaflix.com',
    'https://www.sawaflix.com',
    'http://localhost:3000' // dev only
  ];

  if (allowedOrigins.includes(origin)) {
    const response = NextResponse.next();
    response.headers.set('Access-Control-Allow-Origin', origin);
    response.headers.set('Access-Control-Allow-Methods', 'GET, POST, PUT, DELETE');
    response.headers.set('Access-Control-Allow-Headers', 'Content-Type, Authorization');
    response.headers.set('Access-Control-Max-Age', '86400');
    return response;
  }

  return NextResponse.next();
}
```

### 3. Input Sanitization

```
import DOMPurify from 'isomorphic-dompurify';

// Sanitize any user text input
export function sanitizeInput(input: string): string {
  return DOMPurify.sanitize(input, { ALLOWED_TAGS: [] });
}
```

---

## 📋 GitHub Board Issues to Create

## CRITICAL - Must Fix Before Tuesday

1. **[Backend] File size validation missing in upload endpoint (HIGH)**
    - Affects: Wohking's task
    - Estimated effort: 30 min
    - Blocking: Security
2. **[Backend] Supabase Storage RLS policies not configured (CRITICAL)**
    - Affects: Wohking's task
    - Estimated effort: 1 hour
    - Blocking: Security + Data Privacy
3. **[Backend] Database schema for verification_submissions missing (CRITICAL)**
    - Affects: Wohking's task
    - Estimated effort: 1 hour
    - Blocking: Data Storage
4. **[Backend] Profile table missing role & verification_status columns (CRITICAL)**
    - Affects: Ngam's task
    - Estimated effort: 1 hour
    - Blocking: Auth System
5. **[Backend] OTP is not actually sent via email (CRITICAL)**
    - Affects: Ngam's task
    - Estimated effort: 1.5 hours
    - Blocking: User Experience

## HIGH PRIORITY - Must Fix Before Merge

6. **[Backend] Add rate limiting to upload endpoint (HIGH)**
    - Affects: Wohking's task
    - Estimated effort: 30 min
7. **[Backend] Add rate limiting to OTP verification (HIGH)**

    - Affects: Ngam's task
    - Estimated effort: 30 min
8. **[Backend] Middleware not enforcing role-based access control (HIGH)**
    - Affects: Ngam's task
    - Estimated effort: 1 hour
9. **[Backend] Add virus scanning for uploaded files (HIGH)**
    - Affects: Wohking's task
    - Estimated effort: 2 hours
10. **[Backend] No audit trail for verification events (HIGH)**
    - Affects: Both tasks
    - Estimated effort: 1.5 hours

## MEDIUM PRIORITY - Nice to Have

11. **[Backend] Add file integrity checks (checksums) (MEDIUM)**
    - Affects: Wohking's task
    - Estimated effort: 1 hour
12. **[Backend] Add session invalidation after admin approval (MEDIUM)**
    - Affects: Ngam's task
    - Estimated effort: 1 hour
13. **[Backend] Create integration test suite for verification flow (MEDIUM)**
    - Affects: Both tasks
    - Estimated effort: 2 hours
14. **[Backend] Document admin role assignment process (MEDIUM)**
    - Affects: Ngam's task
    - Estimated effort: 30 min
15. **[Backend] Remove console.log from production code (MEDIUM)**
    - Affects: Both tasks
    - Estimated effort: 30 min

---

# ⏰ Estimated Timeline to Production-Ready

## If Critical Issues Fixed (Best Case)

- Monday 23rd: 6-8 hours of work
- Tuesday 24th: 10 AM presentation - **READY (with limitations)**
- Wednesday 25th: Final polish

## Current Status Without Fixes

- **NOT READY** for production

## What Can Be Presented on Tuesday

If critical MUST-FIX items are completed:

- ✅ OTP flow working end-to-end
- ✅ File upload working with validation
- ✅ Admin approval system working
- ✅ Database schema defined

Limitations (discuss in meeting):

- Virus scanning not yet implemented (future release)
- Checksums not yet added (future release)
- Session refresh needs frontend work (parallel track)

---

# 📝 Recommended Next Steps

## Before Tuesday 10 AM:

**Priority 1 (Wohking) - 3 hours:**

- [ ] Add file size validation
- [ ] Configure RLS policies
- [ ] Create verification_submissions table

**Priority 1 (Ngam) - 3 hours:**

- [ ] Add role & verification_status columns
- [ ] Implement email OTP sending
- [ ] Add OTP verification rate limiting

**Priority 2 (Both) - 2 hours:**

- [ ] Middleware role-based access control
- [ ] Audit logging setup
- [ ] Update backend guide documentation

## For Code Review on Tuesday:

- Come prepared with questions on each issue
- Have mitigation plans for non-blocking issues
- Have implementation PRs ready for merge

---

## 💡 Best Practices Applied Going Forward

1. **Always validate file sizes server-side**
2. **Never expose sensitive info in logs**
3. **Always implement rate limiting on user actions**
4. **Use state machines for status transitions**
5. **Add RLS policies BEFORE going live**
6. **Implement audit trails for sensitive operations**
7. **Use type safety (TypeScript) consistently**
8. **Write integration tests, not just unit tests**
9. **Document security requirements upfront**
10. **Security review before code review**

---

## 📞 Questions for Team Discussion

**For Wohking:**

1. What's your maximum file upload size target?
2. Do you need image optimization (thumbnails)?
3. Should we support batch uploads?
4. How long should files be retained?

**For Ngam:**

1. Should OTP be SMS + Email or Email-only?
2. Should we support passwordless login?
3. Do we need two-factor authentication later?
4. Admin role assignment process - automated or manual?

**For All:**

1. What's the deployment target? (Vercel, self-hosted, other?)
2. Should we use Supabase Edge Functions for sensitive operations?
3. Do we need CDN for storage uploads?
4. Compliance requirements (GDPR, data residency)?

---

## 🎯 Success Criteria for Code Review Pass

✅ **Wohking's Task - PASS when:**

- File size validation implemented
- RLS policies configured and tested
- Audit logging in place

- No sensitive data in logs
- All 10 upload test cases pass

✅ **Ngam's Task - PASS when:**

- Email OTP sending working end-to-end
- Role & verification_status columns added
- Middleware enforces role-based access
- OTP rate limiting works
- All 12 auth test cases pass

**Overall - PASS when:**

- Both sections pass
- Integration tests pass
- Security review passed
- No HIGH severity issues remain

---

**Document Prepared By:** Backend Code Review Team
**Date:** February 21, 2026
**Status:** READY FOR 10 AM TUESDAY PRESENTATION
**Next Review:** Post-implementation (Saturday 27th Feb)

---